

P Praktikum Sisop

- 12524034 - Immanuel Alexander Tambunan
- 12524012 - Chogy Paranda Sitait
- 12524003 - Michael Pratama Nasution

A. Teori

1. 2. Executable File & Bagian Memori:

Executable file adalah berkas yang berisi instruksi mesin yang siap dijalankan oleh CPU. Saat dimuat ke memori (RAM).

in ter bagi menjadi beberapa segmen:

1. Text segment: Berisi instruksi kode program (bersifat read-only)
2. Data segment: Berisi variable global dan statis yang sudah diinisialisasi
3. BSS segment: Berisi variable global dan statis yang belum diinisialisasi.
4. Stack: Menyimpan variable ~~global~~ lokal dan pemanggilan fungsi
5. Heap: Digunakan untuk alokasi memori dinamis saat program berjalan.

b. Langkah Umum Pemuatan Program (Program Loading).

1. Alokasi: Sistem Operasi (OS) membaca header file untuk menentukan kebutuhan memori
2. Pemetaan: OS membuat ruang alamat virtual untuk proses tersebut.
3. Loading: Instruksi dan data dari disk disalin ke RAM.
4. Inisialisasi: Mengatur registrasi dan melompat ke alamat awal program.

c. Process Control Block (PCB):

PCB adalah struktur data pada kernel OS yang menyimpan informasi tentang proses tertentu.

Atribut penting meliputi

1. Process State: Status proses
2. Program Counter: alamat instruksi
3. CPU Register: isi register saat intrupsi
4. PID (Process ID): identitas unik proses

2. Batch System dan Time Shared System

A. Batch System: User tidak berinteraksi langsung dengan komputer. Pekerjaan (jobs) dikelompokkan dengan tipe yang serupa dan dijalankan secara berurutan oleh operator. Fokus utamanya adalah efisiensi penggunaan CPU.

B. Time-Shared System: CPU berpindah antar-proses dengan sangat cepat sehingga user merasa seolah-olah program mereka berjalan sendiri-sendiri secara interaktif. Fokus utamanya adalah waktu respon.

3. Diagram Context Switching (4 Proses)

Dalam context switching, CPU menyimpan keadaan (context) dari proses yang sedang berjalan ke dalam PCB-nya, lalu membuat keadaan proses berikutnya dari PCB-nya.

- Urutannya:
1. P1 berjalan → Simpan konteks P1 → Muat konteks P2
 2. P2 berjalan → Simpan konteks P2 → Muat konteks P3
 3. P3 berjalan → Simpan konteks P3 → Muat konteks P4
 4. P4 berjalan → Simpan konteks P4 → Muat P1 kembali

4. Analisis Kode Program C (Linux Process)

a. Fungsi & Tipe Data

- * `pid_t` : Tipe data khusus untuk menampung ID proses (int)
- * `fork()` : Fungsi untuk membuat proses baru (duplikasi)
- * `wait()` : Fungsi yang memaksa parent berhenti sampai salah satu child-nya selesai dieksekusi

b. Langkah Pengelolaan Process

- * `Value = 100` : inisialisasi variable global
- * `fork()` : sistem memisah menjadi dua proses Parent dan Child
- * Percabangan : Mengecek nilai kembalian `fork()` untuk menentukan jalur eksekusi
- * Eksekusi : Child mengubah value, Parent menunggu (`wait`)

c. Mengapa perlu 3 blok kondisional?

Karena `fork()` mengembalikan tiga kemungkinan nilai yang menentukan status:

1. `< 0` : gagal membuat proses baru (Error)
2. `== 0` : Kode yang akan dijalankan oleh Child Process
3. `> 0` : Kode yang akan dijalankan oleh Parent Process (bersi PID si anak)

d. Tampilan Output (ID induk : 2099, Anak 2095):

Karena ada `wait(NULL)`, maka Child akan selesai lebih dulu, baru kemudian Parent.

- * Baris 16 (Child) : `pid = 0` (dalam Child, `fork()` mengembalikan 0)
- * Baris 17 (Child) : `value = 110 (100 + 10)`
- * Baris 22 (Parent) : `pid = 2095` (dalam Parent, `fork()` mengembalikan PID anak)
- * Baris 23 (Parent) : `value = 100`

e. Modifikasi dengan `exedp` :
ubah bagian dalam `else if (pid == 0)` menjadi

```
else if (pid == 0) {  
    execlp("/bin/sh", "sh", NULL);  
}
```

5. Inter-Process Communication (IPC)

• **Shared Memory** : Dua atau lebih proses berbagi wilayah memori yang sama. Ini sangat cepat karena tidak perlu melibatkan kernel untuk transfer data, namun butuh sinkronisasi agar tidak terjadi konflik

• **Message Passing** : Proses berkomunikasi dengan mengirimkan pesan melalui kernel. Ini lebih mudah diimplementasikan.

(tidak butuh sinkronisasi manual yang rumit dan cocok untuk sistem terdistribusi, namun lebih lambat karena melibatkan system call)